UNIT II

WORD LEVEL ANALYSIS: Unsmoothed N-grams, Evaluating N-grams, Smoothing, Interpolation and Backoff – Word Classes, Part- of-Speech Tagging, Rule-based, Stochastic and Transformation-based tagging, Issues in PoS tagging – Hidden Markov and Maximum Entropy models.

Introduction

Word-level analysis in NLP involves breaking down text into individual tokens to understand their structure, meaning, and grammatical function. Key techniques include tokenization, morphological analysis (root/affix identification), stemming, lemmatization, and POS tagging to enable tasks like sentiment analysis, text classification, and machine translation.

Key Components of Word-Level Analysis:

- **Tokenization:** Dividing text into individual tokens (words, punctuation).
- **Morphological Analysis:** Breaking words into morphemes (roots and affixes) to understand structure (e.g., "playing" →"play" + "ing").
  - **Inflectional:** Changes form, not grammatical category (e.g., "run" to "runs").
  - **Derivational:** Changes grammatical category or meaning (e.g., "run" to "runner").
- **Stemming & Lemmatization:** Reducing words to their base form.
- **Part-of-Speech (POS) Tagging:** Assigning grammatical roles (noun, verb, adjective) to tokens.
- **Regular Expressions (Regex):** Using pattern matching to identify specific word formats, such as dates or email addresses.

Importance and Applications:

Word-level analysis is foundational for, among other things:

- **Simplifying text:** Reducing dimensionality for machine learning models.
- **Handling morphology:** Processing morphologically rich languages by analyzing word roots.
- **Improving accuracy:** Enhancing NLP tasks like sentiment analysis and entity recognition.
- **Text Representation:** Enabling encoding methods like One-Hot Encoding, Bag of Words, and TF-IDF.

Unsmoothed N-grams

Unsmoothed N-grams in NLP are statistical language models that estimate the probability of a word sequence by directly calculating the frequency of contiguous n-word phrases (unigrams, bigrams, etc.) from a training corpus using Maximum Likelihood Estimation (MLE). They are simple, direct representations of word sequences but suffer from the "zero-probability problem"—assigning zero probability to valid sequences not seen in the training data—making them less robust for unseen or rare data, which necessitates smoothing techniques.

Key Aspects of Unsmoothed N-grams:

- **Definition:** N-grams are contiguous sequences of n items (words, letters) in text. Common types include Unigrams (N=1), Bigrams (N=2), and Trigrams (N=3).

- **Calculation:** The probability of a word sequence is estimated by dividing the count of the specific n -gram by the count of its prefix (the preceding n−1words).

- **Zero-Probability Problem:** If a particular word combination, such as "unseen phrase", never appears in the training data, an unsmoothed model assigns it a probability of 0, potentially making the entire sentence probability 0.

- **Data Sparsity:** Because they rely solely on observed data, they struggle with large n-values, where many valid combinations are not represented in the corpus (sparsity).

Advantages:

- **Simplicity:** Easy to calculate and implement.
- **Accuracy on Frequent Events:** Highly accurate for predicting common, frequently observed words in training data.

Disadvantages:

- **Zero-Probability Problem (Sparsity):** If a specific word sequence (e.g., a trigram) does not appear in the training data, the model assigns it a probability of zero, which can ruin applications like spellcheckers or machine translation.
- **Memory Intensive:** Higher order n-grams (e.g., 5-grams) can require huge amounts of storage.

Applications:

- Used in speech recognition, text generation, and spell checking.
- Often serve as a baseline model before applying smoothing (e.g., Laplace, Kneser-Ney) or neural methods.

Unsmoothed models are rarely used in modern production, having been largely superseded by neural network-based language models, but they provide a foundational, computationally efficient approach for basic text analysis and understanding n-gram frequency.

Evaluating N-grams

Evaluating N-gram models in NLP involves measuring how accurately they predict text sequences using intrinsic methods like **perplexity** (minimizing model surprise on test data) or extrinsic methods (evaluating performance within applications like speech recognition or machine translation). Key metrics include ROUGE-N for overlap, with smoothing techniques (e.g., Laplace, Kneser-Ney) used to handle unseen word pairs.

Key Evaluation Methods for N-Grams

- **Intrinsic Evaluation (Perplexity):** The most common metric, measuring how well the probability model predicts a held-out test set. Lower perplexity indicates a better model because it signifies the model is less surprised by the new data.

- **Extrinsic Evaluation:** Tests the N-gram model within a specific application, such as evaluating how well it improves word error rates in speech recognition or fluency in machine translation.

- **ROUGE-N (Recall-Oriented Understudy for Gisting Evaluation):** Used primarily in text summarization, it counts the number of matching N-grams between a system-generated text and human references.

Challenges and Refinements

- **Data Sparsity:** Many valid N-grams do not appear in the training corpus, leading to zero probabilities for those sequences.
- **Smoothing Techniques:** To fix sparsity, smoothing (e.g., Add-one/Laplace, Good-Turing, or Kneser-Ney smoothing) redistributes probability mass to unseen N-grams.
- **Context Limitation:** Larger N (e.g., 4-grams or 5-grams) provides more context but increases the risk of data sparsity, while smaller N (unigrams/bigrams) is more robust but less informative.

Applications in Evaluation

- **Speech Recognition:** Assessing the frequency of word sequences to correct noisy, misinterpreted input.
- **Text Generation:** Comparing the frequency of generated n-grams against human-written text to ensure natural, varied output.
- **Spelling Correction:** Identifying unlikely word combinations, even if individual words are spelled correctly.

Evaluating N-gram models in NLP involves
measuring how accurately a model predicts the next word or sequence, typically using intrinsic metrics like **perplexity** (lower is better) or extrinsic, application-based testing. Key techniques include addressing data sparsity with **smoothing** (e.g., Laplace, Kneser-Ney) to assign non-zero probabilities to unseen sequences.
Key Evaluation Methods for N-grams

- **Perplexity (PP):** The standard intrinsic metric, perplexity measures how well a probability model predicts a sample. A lower perplexity indicates the model is less surprised by the test data, thus predicting it better.
- **Extrinsic Evaluation:** Testing the N-gram model within a specific application, such as speech recognition or machine translation, to see if it improves final output quality.
- **Intrinsic Evaluation:** Measuring the model independently, such as calculating perplexity on a held-out test set.
- **ROUGE-N:** Used primarily in summarization, this measures the overlap of N-grams between system-generated output and human references.

Challenges and Solutions

- **Data Sparsity:** High-order N-grams (e.g., 4-grams) may not appear in training data, leading to zero probabilities. Smoothing techniques, like Laplace or Kneser-Ney smoothing, are essential to adjust probabilities.
- **Model Size:** As 'n' increases, the model size grows exponentially, requiring techniques like pruning or smoothing to manage performance.
- **Context Limitation:** N-grams only consider the last $n-1$words, failing to capture long-distance dependencies.

Evaluating N-Gram Performance

- **Unigrams (n=1):** Treat words as independent, useful for basic frequency analysis.
- **Bigrams (n=2) & Trigrams (n=3):** Model local context, essential for autocomplete and spell-check.
- **Evaluation Results:** Studies show that larger n-grams (n>4) may produce less novel, more repetitive text, indicating a trade-off between fluency and diversity.

Smoothing

Smoothing in NLP adjusts probabilities in statistical models (like n-gram models) to prevent assigning zero probability to unseen words or sequences, ensuring models generalize better by redistributing probability mass from frequent to rare/unseen events, using techniques such as Laplace, Good-Turing, and Kneser-Ney smoothing. This critical step improves performance on real-world data by making models more robust to out-of-vocabulary terms.

Why smoothing is needed

- **Zero Probability Problem:**
  Training data can't contain every possible word or phrase, leading to zero probabilities for unseen combinations, which makes the model fail.
- **Model Robustness:**
  Prevents complete breakdown when encountering new data by giving unseen events a small, non-zero chance.

Common Smoothing Techniques

- **Laplace Smoothing (Add-One):** Adds 1 to every count (numerator and denominator) to give unseen events a small probability, but can drastically alter original probabilities.
- **Add-k Smoothing:** Generalizes Laplace by adding a small constant k instead of 1, reducing the impact.
- **Good-Turing Smoothing:** Estimates the probability of unseen items based on the frequency of items seen only once (n1), effectively discounting seen items to reserve probability mass.
- **Kneser-Ney Smoothing:** An advanced and highly effective method that considers the diversity of contexts in which words appear, not just their frequency, offering better probability distributions for n-grams.
- **Interpolation/Backoff:** Combines models of different orders (e.g., trigram, bigram, unigram) by "backing off" to lower-order models when higher-order data is missing, often using weighted averages.

Smoothing works by taking some probability mass from the frequently occurring n-grams and redistributing it among the less frequent or unseen n-grams, ensuring all probabilities sum to one.

Interpolation and Backoff

Interpolation and backoff are key techniques in Natural Language Processing (NLP) to handle data sparsity in N-gram language models, preventing zero probabilities for unseen word sequences. Interpolation mixes probabilities from multiple N-gram orders (e.g., trigram, bigram, unigram) using weighted sums, while backoff uses lower-order models only when higher-order ones are unavailable.

Key Techniques in Language Modelling

- **Interpolation (Linear or Mixture):** Instead of relying on a single N-gram model, interpolation combines different orders (e.g., $P_{interp} = \lambda_3 P_{tri} + \lambda_2 P_{bi} + \lambda_1 P_{uni}$). It often performs better on small datasets because it uses all available knowledge.
- **Backoff (Simple or Katz):** If an n-gram has zero frequency, the model "backs off" to a lower-order model (e.g., from trigram to bigram).
  - **Katz Backoff:** Uses discounted counts for higher-order n-grams and falls back to lower-order models for unseen data.
  - **Stupid Backoff:** A simpler, efficient method often used for massive datasets (like web-scale data) that skips rigorous probability normalization.
- **Comparison:** Backoff is typically better for large datasets (high counts), whereas interpolation is more effective for small datasets (low counts).

These techniques ensure that language models can generalize better to unseen, but plausible, text combinations.

Word Classes

Word classes (or parts of speech) in Natural Language Processing (NLP) categorize words based on their syntactic function, form, and behavior, with major classes being nouns, verbs, adjectives, and adverbs. Essential for tasks like parsing and sentiment analysis, these classes are divided into open (constantly evolving, e.g., nouns) and closed (stable, e.g., prepositions) sets.

Major Word Classes (Parts of Speech)

Word classes are defined by their grammatical role rather than just meaning, allowing machines to understand sentence structure.

- **Nouns (NN):** Represent people, places, things, or concepts (e.g., "algorithm", "data").
- **Verbs (VB):** Express actions or states of being (e.g., "process", "analyzed").
- **Adjectives (JJ):** Describe or modify nouns (e.g., "efficient", "relevant").
- **Adverbs (RB):** Modify verbs, adjectives, or other adverbs (e.g., "quickly", "hardly").

Classification of Word Classes

NLP further categorizes these into two main types:

- **Open Class (Content Words):** These classes accept new words (e.g., "tweet" as a verb/noun) and include nouns, verbs, adjectives, and adverbs.
- **Closed Class (Function Words):** These are stable, structural words including pronouns, prepositions, determiners, and conjunctions.

Role in NLP: POS Tagging

Part of Speech (POS) tagging is the process of marking up a word in a text to a particular part of speech based on its definition and context.

- **Techniques:** Approaches include rule-based systems, statistical methods (Hidden Markov Models), and deep learning techniques (Recurrent Neural Networks).
- **Applications:** Essential for information extraction and machine translation to understand that "bank" in "river bank" is a noun, not a verb.

Part- of-Speech Tagging

POS (Part-of-Speech) tagging in NLP is the process of assigning grammatical categories (like noun, verb, adjective) to each word in a text, helping machines understand sentence structure and meaning by identifying each word's role, crucial for applications like machine translation and sentiment analysis. It works by using context to disambiguate words (e.g., "book" as a verb vs. a noun) and uses methods like rule-based, statistical, or HMM models, with popular tagsets like the Penn Treebank.

How it works

- **Tokenization**: First, the text is broken down into individual words (tokens).
- **Contextual analysis**: A tagger analyzes surrounding words to determine the correct part of speech for ambiguous words.
- **Tag assignment**: Each word receives a tag, creating a list of (word, tag) pairs.
  Ex: "Book the flight" becomes [("Book", "Verb"), ("the", "Det"), ("flight", "Noun")].

Key Concepts

- **Tags**: Labels for grammatical roles (e.g., NN for Noun, VB for Verb, JJ for Adjective).
- **Tagsets**: Standardized collections of tags, with the Penn Treebank being very common for English.

Common Methods

- **Rule-Based**: Uses predefined linguistic rules.
- **Statistical**: Learns from labeled data (e.g., Hidden Markov Models - HMM).
- **Transformation-Based**: Uses transformation rules to refine initial tags.

Why it's important

- **Structural Understanding**: Reveals how words function together in a sentence.
- **Foundation for Other Tasks**: Essential for Named Entity Recognition (NER), information retrieval, and machine translation.

Rule – Based

Rule-based Natural Language Processing (NLP) is an approach that uses manually created linguistic rules to help computers understand and process human language. In this method, domain experts define explicit **"if-then" statements** and pattern-matching techniques to analyze text, unlike modern machine learning methods that learn patterns automatically from data.

How it Works

The rule-based approach involves several stages:

1. **Rule Creation:** Linguists and domain experts create specific rules, grammar, and lexicons (dictionaries) to capture linguistic structures, such as identifying nouns or verbs in a sentence.
2. **Rule Application:** The predefined rules are applied to the input text data to match patterns.
3. **Rule Processing:** The text is processed according to the matched rules to extract information or make decisions (e.g., categorizing a word based on its context).
4. **Rule Refinement:** The rules are iteratively refined and updated based on the system's performance and feedback to improve accuracy.

Advantages:

- **Easily interpretable and explainable** because the rules are explicitly defined.
- **High precision** for specific, well-defined problem sets and controlled language (e.g., legal documents).
- **Works with limited or no training data**, making it useful in data-scarce domains.
- **Fast computation time** during execution as it primarily involves pattern matching.

Disadvantages

- **Labor-intensive and time-consuming** to create and maintain rules manually.
- **Poorly handles variations, exceptions, and ambiguity** in natural, everyday language.
- **Does not scale well** to complex or large-scale applications due to the difficulty of creating enough rules.
- **Requires continuous manual updates** to accommodate evolving language use and new words.

While machine learning and deep learning (especially large language models like GPT-4o) have become the dominant NLP approaches due to their flexibility and scalability, rule-based methods are still valuable.

They are often used in **hybrid models** to handle specific, highly structured tasks (like URL or date format recognition), to speed up the initial annotation of training data for machine learning models, or in fields requiring high precision and explainability, such as clinical decision support systems and legal text analysis. Tools like the spaCy library offer rule-based matching engines that can work alongside deep learning models.

Stochastic and Transformation - Based Tagging

Stochastic and transformation-based tagging are two prominent approaches in Natural Language Processing (NLP) for assigning Parts-of-Speech (POS) tags to text. Stochastic tagging uses probability models (like Hidden Markov Models) to choose the most likely tag based on frequencies. Transformation-based tagging (Brill tagging) applies learned, ordered rules to iteratively fix initial errors, combining statistical learning with rule-based interpretability.

Stochastic Tagging (Probabilistic)

- **Definition:** Relies on frequency, probability, and statistics derived from a trained annotated corpus to identify the most likely sequence of tags.

- **Mechanism:** It often chooses the most frequently used tag for a word in the training data (e.g., if "play" is mostly a noun, it labels it as such).

- **Common Models:** Hidden Markov Model (HMM), N-gram taggers.
- **Pros:** Generally achieves high accuracy.
- **Cons:** Can yield invalid tag sequences that do not follow grammatical rules

Transformation-Based Tagging (Brill Tagging)

- **Definition:** A rule-based, error-driven learning approach that starts with an initial state (often using a simple stochastic approach) and improves it iteratively by applying learned transformation rules.

- **Mechanism:** Learns rules from training data, such as "Change a tag from verb to noun if the previous word is a determiner". It applies these rules in a specific order to correct mistakes.

- **Pros:** Highly interpretable (rules are readable), relatively fast, and often requires a smaller, more focused set of rules.

- **Cons:** Training process can be lengthy, and it does not inherently provide probabilities.

Comparison

Feature	Stochastic Tagging	Transformation-Based Tagging
Foundation	Probabilities/Statistics	Rule-based (Error-driven)
Approach	Finds maximum probability	Iterative refinement
Interpretability	Low (Black box)	High (Readable rules)
Speed	Generally faster	Faster at inference

Issues in PoS Tagging

Part-of-Speech (PoS) tagging in NLP faces significant challenges primarily due to lexical ambiguity, where words possess multiple tags based on context (e.g., "book" as a noun or verb). Other major issues include handling out-of-vocabulary (OOV) words, domain dependence, and the need for large, accurately annotated datasets.

Key issues in PoS tagging include:

- **Lexical Ambiguity (Polysemy):** Many words in English and other languages have multiple meanings and, consequently, multiple possible POS tags depending on context (e.g., "fast" can be an adjective, adverb, noun, or verb).
- **Out-of-Vocabulary (OOV) Words:** Taggers struggle with words not present in their training data, such as new, technical, or misspelled words.
- **Context Dependence:** Accurately tagging requires analyzing the surrounding words and syntactic structure, which is difficult for complex sentences.
- **Domain Dependence:** Models trained on specific data (e.g., news articles) often perform poorly on different domains (e.g., social media or medical records).
- **Data Scarcity and Quality:** The lack of large, annotated training datasets for certain languages or domains makes it difficult to train accurate models, while existing datasets may contain inconsistencies.
- **Morphological Complexity:** Languages with high inflectional variations (e.g., Sanskrit, German) pose greater challenges in mapping word forms to tags compared to languages with rigid word orders.

These challenges are addressed using various approaches, ranging from traditional statistical models (HMM, Maximum Entropy) to deep learning methods (BiLSTM, Transformers).

Hidden Markov Model

A Hidden Markov Model (HMM) in NLP predicts hidden linguistic structures (like Part-of-Speech tags) from visible text (words) by modeling sequences using hidden states (tags), observable outputs (words), transition probabilities (tag-to-tag), and emission probabilities (tag-to-word), effectively solving tasks like POS tagging, named entity recognition, and speech recognition, though often surpassed by deep learning now.

How it works

- **Hidden States (Q):** The underlying, unobservable categories (e.g., Noun, Verb, Adjective).
- **Observations (V):** The actual words in the text (e.g., "The", "cat", "sat").
- **Transition Probabilities (A):** The likelihood of moving from one hidden state to another (e.g., P(Noun | Determiner)).
- **Emission Probabilities (B):** The likelihood of a specific word being generated from a hidden state (e.g., P("cat" | Noun)).
- **Markov Property:** Assumes the next state depends only on the current state, not the entire past.

Key Applications in NLP

- **Part-of-Speech (POS) Tagging:** Inferring if a word is a Noun, Verb, etc., from the word sequence.
- **Speech Recognition:** Mapping acoustic signals (observations) to phonemes/words (hidden states).
- **Morphological Analysis:** Understanding word structure.
- **Named Entity Recognition (NER):** Identifying names, locations, etc..

Core Algorithms

- **Viterbi Algorithm:** Finds the most probable sequence of hidden states (e.g., best POS tag sequence) for a given observation sequence (text).
- **Forward-Backward Algorithm:** Calculates probabilities related to hidden states.

Limitations

- The strong Markov assumption (only current state matters) may not hold for complex language.
- Can be computationally intensive with many states.
- Modern deep learning models often outperform HMMs for complex NLP tasks.

Maximum Entropy Model

A Maximum Entropy (MaxEnt) model in NLP is a powerful probabilistic classifier that selects the most uniform probability distribution (highest entropy) that still satisfies constraints from training data, effectively combining multiple features (like word endings, context words) without assuming independence, making it great for tasks like POS tagging, text classification, and speech recognition by assigning weights

to features to predict linguistic classes. It's a log-linear model, generalizing logistic regression, that finds the best model by maximizing uncertainty (entropy) given observed evidence.

Core Principles & How it Works

- **Principle of Maximum Entropy**:
  Choose the model that best reflects the data but makes the fewest assumptions, essentially distributing probability as evenly as possible.
- **Feature-Based**:
  Uses various input features (e.g., previous words, word suffixes) to predict an output (e.g., a part-of-speech tag).
- **Log-Linear Model**:
  Calculates a weighted sum of features, then uses a normalization constant to get probabilities, making it powerful for interdependent features.
- **Discriminative**:
  Models the conditional probability P(Class | Features) directly.

Key Applications in NLP

- **Part-of-Speech (POS) Tagging**: Determining if "flies" is a noun or verb by using features like surrounding words.
- **Text Classification**: Spam detection (features: sender, keywords, domain) or sentiment analysis.
- **Speech Recognition**: Modeling word sequences and context.
- **Named Entity Recognition**: Identifying names, places, etc., using rich contextual features.

Advantages

- **Handles Feature Dependence**: Unlike models that assume independence (like some Hidden Markov Models), MaxEnt excels with correlated features.
- **Rich Feature Sets**: Allows domain experts to design many powerful, context-specific features.
- **Flexibility**: Can be extended to sequence models like Maximum Entropy Markov Models (MEMMs) for sequential data.

In essence, MaxEnt provides a robust framework to combine diverse linguistic evidence into a single, powerful predictive model.

Comparison

Feature	Hidden Markov Model (HMM)	Maximum Entropy Model (MaxEnt/MEMM)
Approach	Generative	Discriminative
Probability	Joint P(Tag, Word)	Conditional $P(\text{Tag})
Features	Strict independence assumption	Non-independent, rich features
Context	Limited	Broad context possible
Training	Fast	Slow (Iterative optimization)
Label Bias	No	Yes

Both models are foundational in NLP, with MaxEnt often serving as a more flexible, discriminative alternative to the strictly probabilistic HMM.